DBS101-Lab 04

Date: 02.04.2026

Aim: Creating database users and granting permissions on database, tables, views, and row-level security.

LAB TASK

A company wants to manage access to its database by creating different users and granting permissions based on their roles. The database contains department and employee information. Different users should have different access rights.

Create and use a database for this scenario, in a format similar to the uploaded lab

Table 1: department

Store the following details:

- dept_id – integer, primary key
- dept_name – variable character, maximum 50 characters, must not be null
- location – variable character, maximum 50 characters

Table 2: employee

Store the following details:

- emp_id – integer, primary key
- emp_name – variable character, maximum 50 characters, must not be null
- salary – decimal value
- dept_id – integer, foreign key referencing department(dept_id)

Tasks

1. Create a database named security_db.

```
CREATE DATABASE security_db;
```

2. Connect to the security_db database.

```
\c security_db;
```

3. Create the department table with the required constraints.

```
CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL,
    location VARCHAR(50)
);
```

4. Create the employee table with the required constraints and foreign key.

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

5. Insert the following records into the department table.

```
INSERT INTO department (dept_id, dept_name, location) VALUES
(1, 'Human Resource', 'Block A'),
(2, 'IT', 'Block B'),
(3, 'Finance', 'Block C');
```

6. Insert the following records into the employee table.

```
INSERT INTO employee (emp_id, emp_name, salary, dept_id) VALUES
(101, 'Asha', 30000, 1),
(102, 'Tashi', 45000, 2),
(103, 'Karma', 50000, 2),
(104, 'Sonam', 40000, 3),
(105, 'Pema', 35000, 1);
```

User Creation Tasks

7. Create a database user named admin_user with a password.

```
CREATE USER admin_user WITH PASSWORD 'admin123';
```

8. Create a database user named hr_user with a password.

```
CREATE USER hr_user WITH PASSWORD 'hr123';
```

9. Create a database user named manager_user with a password.

```
CREATE USER manager_user WITH PASSWORD 'manager123';
```

10. Create a database user named finance_user with a password.

```
CREATE USER finance_user WITH PASSWORD 'finance123';
```

11. Display the list of created users.

```
\du
```

Permission Tasks: Database Level

12. Grant all privileges on the security_db database to admin_user.

```
GRANT ALL PRIVILEGES ON DATABASE security_db TO admin_user;
```

13. Grant permission to connect to the database for hr_user, manager_user, and finance_user.

```
GRANT CONNECT ON DATABASE security_db TO hr_user, manager_user, finance_user;
```

14. Grant usage on schema public to all created users.

```
GRANT USAGE ON SCHEMA public TO hr_user, manager_user, finance_user, admin_user;
```

Permission Tasks: Table Level

15. Grant SELECT permission on the employee table to hr_user.

```
GRANT SELECT ON employee TO hr_user;
```

16. Grant SELECT and INSERT permissions on the employee table to manager_user.

```
GRANT SELECT, INSERT ON employee TO manager_user;
```

17. Grant SELECT permission on the department table to hr_user and manager_user.

```
GRANT SELECT ON department TO hr_user, manager_user;
```

18. Grant SELECT permission on the employee table to finance_user.

```
GRANT SELECT ON employee TO finance_user;
```

19. Revoke INSERT permission on the employee table from manager_user.

```
REVOKE INSERT ON employee FROM manager_user;
```

View Tasks

20. Create a view named employee_basic_view to display emp_id, emp_name, and dept_id.

```
CREATE VIEW employee_basic_view AS
SELECT emp_id, emp_name, dept_id
FROM employee;
```

21. Grant SELECT permission on employee_basic_view to hr_user.

```
GRANT SELECT ON employee_basic_view TO hr_user;
```

22. Create a view named finance_view to display emp_id, emp_name, and salary.

```
CREATE VIEW finance_view AS
SELECT emp_id, emp_name, salary
FROM employee;
```

23. Grant SELECT permission on finance_view to finance_user.

```
GRANT SELECT ON finance_view TO finance_user;
```

24. Display all records from both created views.

```
SELECT * FROM employee_basic_view;
SELECT * FROM finance_view;
```

Row-Level Security Tasks

25. Enable row-level security on the employee table.

```
ALTER TABLE employee ENABLE ROW LEVEL SECURITY;
```

26. Create a policy so that hr_user can view only employees belonging to department 1.

```
CREATE POLICY hr_policy ON employee
FOR SELECT
TO hr_user
USING (dept_id = 1);
```

27. Create a policy so that finance_user can view only employees belonging to department 3.

```
CREATE POLICY finance_policy ON employee
FOR SELECT
TO finance_user
USING (dept_id = 3);
```

28. Create a policy so that manager_user can view all rows.

```
CREATE POLICY manager_policy ON employee
FOR SELECT
TO manager_user
USING (true);
```

Permission Verification Tasks

29. Log in as hr_user and test whether the user can read from employee_basic_view.

```
SET ROLE hr_user;
SELECT * FROM employee_basic_view;
RESET ROLE;
```

30. Log in as hr_user and test whether the user can read from department.

```
SET ROLE hr_user;
SELECT * FROM department;
RESET ROLE;
```

31. Log in as hr_user and test whether the user can update the employee table.

```
SET ROLE hr_user;
UPDATE employee SET salary = 32000 WHERE emp_id = 101;
RESET ROLE;
```

32. Log in as manager_user and test whether the user can read from employee.

```
SET ROLE manager_user;
SELECT * FROM employee;
RESET ROLE;
```

33. Log in as manager_user and test insert permission after revoke.

```
SET ROLE manager_user;
INSERT INTO employee VALUES (106, 'Dechen', 38000, 2);
RESET ROLE;
```

34. Log in as finance_user and test whether the user can read from finance_view.

```
SET ROLE finance_user;
SELECT * FROM finance_view;
RESET ROLE;
```

35. Log in as finance_user and test row-level access on employee table.

```
SET ROLE finance_user;
SELECT * FROM employee;
RESET ROLE;
```

SQL Query Tasks

36. Display all records from the department table.

```
SELECT * FROM department;
```

37. Display all records from the employee table.

```
SELECT * FROM employee;
```

38. Display all records from employee_basic_view.

```
SELECT * FROM employee_basic_view;
```

39. Display all records from finance_view.

```
SELECT * FROM finance_view;
```

40. Show the privileges granted on the employee table.

```
\z employee
```

41. Show the privileges granted on the views.

```
\z employee_basic_view
\z finance_view
```

42. Show the row-level security policies created in the database.

```
SELECT * FROM pg_policies WHERE tablename = 'employee';
```

Final Output

43. Display the final contents of all tables.

```
SELECT * FROM department;
SELECT * FROM employee;
```

44. Display the created views.

```
SELECT * FROM employee_basic_view;
SELECT * FROM finance_view;
```

45. Display the created users and their roles.